Multi-Product Shopping Budget Constraint

Problem statement

Evaluate a basket of products and quantities, apply discount conditions, and determine which quantities must be reduced to fit a fixed budget.

Assumptions and rules

• Each product record contains name, unit price, quantity, and discount percentage.

• A 10% discount applies to an individual product line when its quantity is at least 3.

• If the basket exceeds budget, quantities are reduced one unit at a time, starting with the most expensive unit price, until the total is within budget.

• The program does not increase quantities.

Algorithm

1. Read budget and product records.

2. Calculate each line's subtotal and apply the quantity-based discount.

3. Calculate the basket total.

4. If the total is over budget, repeatedly find the product with the highest unit price and quantity above zero.

5. Reduce that quantity by one and recalculate the total and discounts.

6. Stop when the total is within budget or no products remain.

7. Display original and adjusted quantities, savings, and final total.

Pseudocode

```
START\nINPUT budget and basket\nCALCULATE total using line discounts\nWHILE total > budget AND some quantity > 0\n  FIND pr
```

Flowchart

```
[Start] -> [Read budget and basket]\n   |\n[Calculate discounts and total]\n   |\n{Within budget?} --Yes--> [Print basket]\
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
budget = 1000
basket = [
    ["Notebook", 80, 4],
    ["Pen Set", 150, 3],
    ["Water Bottle", 250, 2],
    ["Folder", 60, 5]
]
# Each product: name, unit price, quantity

original_quantities = []
for item in basket:
    original_quantities.append(item[2])

def calculate_total(items):
    total = 0
    for item in items:
        name, price, quantity = item
        line_total = price * quantity
        if quantity >= 3:
            line_total = line_total * 0.90
        total = total + line_total
    return total

original_total = calculate_total(basket)
```

```
print("Original total:", round(original_total, 2))

while calculate_total(basket) > budget:
    highest_index = -1
    highest_price = -1
    for i in range(len(basket)):
        if basket[i][2] > 0 and basket[i][1] > highest_price:
            highest_price = basket[i][1]
            highest_index = i

    if highest_index == -1:
        break

    basket[highest_index][2] = basket[highest_index][2] - 1

print("ADJUSTED BASKET")
for i in range(len(basket)):
    print(basket[i][0], "Original quantity:", original_quantities[i],
          "New quantity:", basket[i][2])

final_total = calculate_total(basket)
print("Final total:", round(final_total, 2))
print("Savings:", round(original_total - final_total, 2))
if final_total <= budget:
    print("Budget condition: SATISFIED")
else:
    print("Budget condition: NOT SATISFIED")
```

Sample output

```
Original total: 1512.0\nADJUSTED BASKET\nNotebook Original quantity: 4 New quantity: 4\nPen Set Original quantity: 3 New qu
```

Explanation / observations

The reduction rule removes one unit at a time from the most expensive product that still has stock. The program recalculates discounts after each change and stops once the basket fits the budget.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.